

Functions

Introduction to Programming

Rae Harbird, r.harbird@ucl.ac.uk

Functions

Questions

Functions

Overview

- ▶ Functions as Black Boxes.
- ▶ Parameter Passing.
- ▶ Return Values.
- ▶ Functions without Return Values.
- ▶ Variable Scope.

Functions as Black Boxes

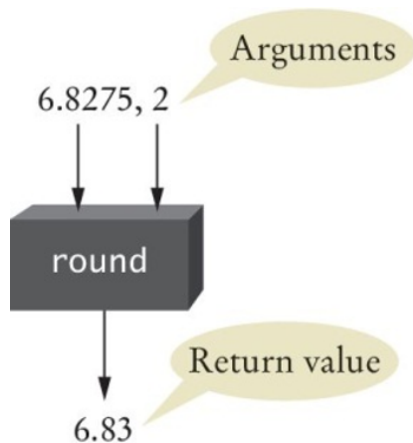


Image: Chapter 5, Python for Everyone, 2nd Ed.

Using Functions

- ▶ Python has built-in functions.
- ▶ Pass the function what it needs to do its job.
- ▶ Use the result

```
# Sets cupVolume to 7.94  
cupVolume = round(7.93627, 2)
```

```
# Get the user's age  
usersAge = int(input("Enter your age: "))
```

You can write ... and use, your own.

Example With a Function

Try the version of the Retirement Age program which uses a function: `src/retire_with_functions.py`

Parameter Passing

Try the version of the program `guess_two_numbers.py` with no functions: `src/guess_two_numbers.py`

Now look at the version with a function that has parameters called `guess_two_numbers_again.py`.

```
def guessesOk(answerA, answerB, guessA, guessB) :  
    okSameOrder = guessA == answerA and guessB == answerB  
    okOtherOrder = guessA == answerB and guessB == answerA  
    return okSameOrder or okOtherOrder  
  
...  
  
if guessesOk(firstAnswer, secondAnswer, firstGuess,  
              secondGuess):  
    println("\tCorrect - well done!\n")  
else:  
    print("\tNo, the answers were {} and {}. \n"  
          .format(firstAnswer, secondAnswer))
```

Notes

- ▶ Parameter variables receive the argument values supplied in the function call
- ▶ The argument value may be:
 - ▶ The contents of a variable
 - ▶ A 'literal': "catch", 22
- ▶ The parameter variable is:
 - ▶ Declared in the called function
 - ▶ Initialised with the value of the argument value
 - ▶ Used as a variable inside the called function

Common Mistakes I

Parameter variables are local to the function, **do not try to change them.**

See: `bad_twice_number_with_functions.py`

```
def twice_number(the_number) :  
    the_number = the_number * 2  
  
def main():  
    the_number = int(input("Enter a value for 'number': "))  
    print("The value of 'number' is {}".format(the_number))  
    twice_number(the_number)  
    print("The value of 'twice_number' is: {}".format(the_number))  
  
if __name__ == "__main__":  
    main()
```

Common Mistakes II

If your function returns a value, make sure it **always** returns a value for every branch of your code.

```
# Yes, please!!!  
def cubeVolume(sideLength) :  
    if sideLength >= 0  
        return sideLength ** 3  
    else :  
        return 0
```

```
# No, No, No  
def cubeVolume(sideLength) :  
    if sideLength >= 0 :  
        return sideLength ** 3  
    # Error-no return value if sideLength < 0
```

Scope

A function may contain local variables. Local parameters are local variables, for example.

A variable can be 'seen' inside the block in which it is declared. The scope of a variable is the part of the program in which it is visible, this means that you can have variables in different functions with the same name. Think of towns which have a Main Street.



(left) © jchamp/iStockphoto; (middle) © StevenCarrieJohnson/iStockphoto; (right) © jsmith/iStockphoto.

Image: Chapter 5, Python for Everyone, 2nd Ed.

Questions

Question 1: True or False?

- a. A function has exactly one return statement.
- b. A function has at least one return statement.
- c. A function has at most one return value.
- d. A function that does not return a value never has a return statement.
- e. When executing a return statement, the function exits immediately.
- f. A function that does not return a value must print a result.
- g. A function without parameter variables always returns the same value.

(Horstmann, p302)

Horstmann, Cay S., Rance Necaise. Python for Everyone, Interactive Edition, 2nd Edition. Wiley, 2016-05-09. VitalBook file.

Answer Q1

- a. A function has exactly one return statement. **False** - a function may have 0 or more return statements.
- b. A function has at least one return statement. **False** - a function may have 0 or more return statements.
- c. A function has at most one return value. **True**
- d. A function that does not return a value never has a return statement. **False** - a function that does not return a value can use a return statement to terminate its execution.
- e. When executing a return statement, the function exits immediately. **True**
- f. A function that does not return a value must print a result. **False** - a function could do anything (or nothing) else instead.
- g. A function without parameter variables always returns the same value. **False** - consider, for example, a function that returns a random integer.

Question 2

Consider these functions:

```
def f(x) :  
    return g(x) + math.sqrt(h(x))
```

```
def g(x) :  
    return 4 * h(x)
```

```
def h(x) :  
    return x * x + k(x) - 1
```

```
def k(x) :  
    return 2 * (x + 1)
```

Question 2 (cont.)

Without actually compiling and running a program, determine the results of the following function calls:

1. $x1 = f(2)$

2. $x2 = g(h(2))$

3. $x3 = k(g(2) + h(2))$

4. $x4 = f(0) + f(1) + f(2)$

5. $x5 = f(-1) + g(-1) + h(-1) + k(-1)$

Answer Q2

1. $x_1 = f(2)$

2. $x_2 = g(h(2))$

3. $x_3 = k(g(2) + h(2))$

4. $x_4 = f(0) + f(1) + f(2)$

5. $x_5 = f(-1) + g(-1) + h(-1) + k(-1)$

The value of x_1 will be 39.0.

The value of x_2 will be 400.

The value of x_3 will be 92.

The value of x_4 will be 62.0.

The value of x_5 will be 0.0.